



What other feature does SORCE and Sprout have other than the following:☒ User registration, authentication, and profile management.

- ☒ Resume upload and management.
- ☒ Job posting management for recruiters.
- ☒ Job search and filtering capabilities.
- ☒ Job application submission.
- ☒ Application status tracking (Applied, Interview, Offered, Rejected).
- ☒ Application history dashboard.
- ☒ Notifications and reminders for application deadlines.
- ☒ Saved jobs and favourites.
- ☒ AI-assisted resume feedback or job matching suggestions (optional advanced feature).

Thanks—those URLs confirm the intended products. Beyond your existing list, the most important additional features are **swipe-to-apply, automated completion of external employer forms, customized resumes and cover letters per job, and application review controls**; Sprout also offers deeper email and document management. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#)

Additional features

Feature	SORCE	Sprout
Swipe-right-to-apply interface	Yes	Yes
AI agent applying on employer websites	Yes	Yes
Automatic ATS form filling	Yes	Yes
Tailored cover letter per job	Yes	Yes
Tailored resume per job	Yes	Yes
Review/edit before submission	Limited; user selects each job	Yes, including optional required approval
Guided job-specific questions	Not confirmed	Yes
Application-linked recruiter inbox	Not confirmed	Yes
Automatic email classification	Not confirmed	Yes
Private email relay	Not confirmed	Yes
Exact document record per application	Not confirmed	Yes
Detailed application timeline	Basic tracking	Yes

Feature	SORCE	Sprout
Automatic resume updating	Not confirmed	Yes
Application performance insights	Not confirmed	Yes
Web and mobile synchronization	iOS-first	Yes
Failed-application recovery	User is notified and can apply manually	Not clearly documented

SORCE's automation can navigate common employer systems, reuse saved information, generate application documents, and submit applications after the user swipes right. Sprout adds guided questions, editable application materials, approval settings, application-specific inboxes, email-based status detection, document storage, and synchronized web/mobile access. [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[1\]](#)

Recommended requirements

Consider adding these features to your requirements:

- Swipe-based job discovery
- AI-powered application-form autofill
- External ATS integration and automated submission
- Job-specific resume and cover-letter generation
- Resume, cover-letter, and answer review before submission
- Optional "Require Approval" setting
- Guided questions to uncover relevant experience
- Application-specific recruiter inbox
- Automatic classification of interview, offer, rejection, and follow-up emails
- Private email forwarding and reply routing
- Document version history for each application
- Chronological application activity timeline
- Duplicate-application detection
- Failed-submission explanations and manual recovery
- Cross-device synchronization
- Application outcome analytics
- Subscription or application-credit management
- Data encryption, consent controls, and audit logs

One correction: **recruiter job-posting management does not appear to be a core feature of either candidate-facing product at these URLs.** Both primarily aggregate external vacancies and submit candidate applications to employer career sites. [\[6\]](#) [\[1\]](#) [\[2\]](#)

What languages is the web version built in? and if you had to rebuild it as a project with no budget what are all the requirements needed in terms of front end, back end, servers, API? And if you were to start with only a few features, what are the top 5 features you should implement?

Both web applications appear to use a modern JavaScript ecosystem. SORCE publicly confirms TypeScript/Node.js, Python and PostgreSQL in its engineering stack; its public website is delivered through Next.js/Vercel. Sprout’s marketing site is built with Framer, while its authenticated web application is delivered as a Next.js application on Vercel; Sprout also discloses Supabase, Trigger.dev and Google Cloud services. ^[43] ^[44] ^[45]

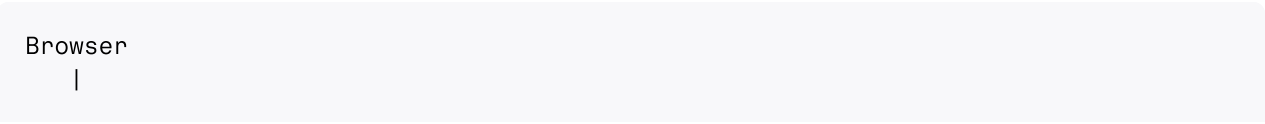
Observed technology

Product surface	Technology identified	Confidence
SORCE website	Next.js/React, JavaScript or TypeScript, HTML and CSS, hosted on Vercel	High from public deployment; exact source configuration is private
SORCE backend	TypeScript/Node.js, Python and PostgreSQL; Go and Rust are also listed as relevant engineering skills	High from SORCE’s YC engineering listing ^[43]
Sprout marketing site	Framer, which produces a React/JavaScript-based website	High ^[44]
Sprout web application	Next.js/React hosted on Vercel; likely TypeScript, although the exact source language cannot be conclusively verified publicly	High for Next.js/Vercel; medium for TypeScript
Sprout data services	Supabase/PostgreSQL for application-email data	Confirmed ^[45]
Sprout background processing	Trigger.dev	Confirmed ^[45]
Sprout AI services	OpenAI, Anthropic, Google Cloud AI and Vertex AI	Confirmed ^[45]
Sprout email integration	Gmail API with read-only OAuth access	Confirmed ^[45]

A deployed site can reveal its framework and hosting signatures, but it cannot reliably reveal every internal language, library or service. Therefore, these should be treated as **publicly observable or disclosed technologies**, not complete source-code inventories.

Recommended architecture

For a zero-budget student or portfolio project, use a **modular monolith** rather than separate microservices:



```
Next.js application
|
Server actions / REST API
|
Supabase
|-- Authentication
|-- PostgreSQL database
|-- Resume file storage
|-- Row-level security
|
External services
|-- Job-listing API
|-- Optional Gemini API
|-- Transactional email service
```

Next.js is suitable because it handles React interfaces, routing, server-side rendering, data fetching and server endpoints in one project. Supabase's free plan combines PostgreSQL, authentication, storage, APIs and server functions, making it practical for a prototype without maintaining a database server. ^[46] ^[47] ^[48]

Front-end requirements

Core stack

- **Language:** TypeScript
- **Framework:** Next.js with React
- **Styling:** Tailwind CSS
- **Components:** shadcn/ui or another free accessible component library
- **Form handling:** React Hook Form
- **Validation:** Zod
- **Server-state management:** TanStack Query, or Next.js server actions for a smaller project
- **Icons:** Lucide
- **Testing:** Vitest and React Testing Library
- **End-to-end testing:** Playwright
- **Package manager:** npm, pnpm or Bun

Required pages

- Landing page
- Registration and sign-in
- Onboarding
- Candidate profile
- Resume upload and editing
- Job feed

- Job search and filters
- Individual job details
- Saved and dismissed jobs
- Application creation
- Application tracker
- Application details and timeline
- Account and privacy settings
- Administrator import/monitoring page

Reusable components

- Navigation bar and mobile menu
- Job card
- Swipe or apply/save/dismiss controls
- Search and filter panel
- Resume uploader
- Profile completion indicator
- Application status board
- Status badge
- Document preview
- Confirmation dialog
- Toast notifications
- Loading skeletons
- Error states
- Empty states
- Pagination or infinite scrolling

User-experience requirements

- Responsive mobile and desktop layouts
- Keyboard-accessible navigation
- Proper labels and focus indicators
- Screen-reader-compatible controls
- Clear validation messages
- Confirmation before destructive actions
- Protection against duplicate applications
- Autosaving forms

- Loading and progress indicators
- Graceful handling of unavailable APIs
- English first, with architecture ready for French localization because the project may serve Montréal or Quebec users

Back-end requirements

Recommended stack

Use Next.js server actions and route handlers as the initial backend. Do not build a separate Express or Python server until a feature genuinely requires it.

Use:

- TypeScript and Node.js
- Supabase PostgreSQL
- Supabase Authentication
- Supabase Storage
- Drizzle ORM, Prisma, or the Supabase JavaScript client
- Zod for server-side request validation
- Scheduled functions only for job imports and reminders
- Python later for advanced resume parsing or ranking experiments

Database tables

Table	Purpose
users	Authentication identity
profiles	Candidate name, location, experience and preferences
skills	Normalized skill definitions
profile_skills	Candidate-to-skill relationships
resumes	Resume metadata and storage path
resume_versions	Original and tailored resume versions
companies	Employer information
jobs	Imported vacancies
job_skills	Skills required by each vacancy
job_interactions	Viewed, saved, liked or dismissed
applications	Candidate's application record
application_events	Status-change timeline

Table	Purpose
cover_letters	Generated or manually written letters
notifications	In-app alerts and reminder records
search_preferences	Roles, locations, salary and work arrangement
ai_generations	AI request type, status and generated-document reference
audit_logs	Security-sensitive operations
api_import_runs	Job-import results and errors

Application status model

Use a controlled set such as:

- Draft
- Ready to apply
- Applied
- Recruiter screening
- Interview
- Assessment
- Offer
- Rejected
- Withdrawn
- Archived

Store status changes separately in `application_events`; do not overwrite all history when an application moves to a new stage.

Back-end services

- Account creation and session management
- Profile CRUD operations
- Resume upload, validation and deletion
- Resume text extraction
- Job ingestion and deduplication
- Search and filtering
- Saved/dismissed job management
- Application CRUD operations
- Status-transition validation
- Application timeline creation

- Notification scheduling
- AI document generation
- Rate limiting
- Audit logging
- Account data export
- Account and file deletion

Server requirements

You do **not** need to purchase or administer a traditional server for the MVP.

Layer	Free option	Responsibility
Source control	GitHub	Repository, issues and pull requests
Front end/server endpoints	Vercel Hobby	Next.js deployment and preview builds
Database	Supabase Free	PostgreSQL
Authentication	Supabase Auth	Email/password, magic link or OAuth
Resume storage	Supabase Storage	PDF and DOCX files
Scheduled jobs	GitHub Actions, Supabase scheduled functions or cron	Periodic job imports and reminders
Monitoring	Vercel logs and Sentry free tier	Runtime errors
Analytics	PostHog free tier or privacy-friendly basic analytics	Feature usage
Email	Resend free tier or Supabase email for development	Verification and notifications
AI	Gemini free tier or local Ollama during development	Resume and cover-letter generation
Domain	Free platform subdomain	Avoid purchasing a custom domain initially

Vercel’s Hobby plan provides free deployment, CI/CD, HTTPS, preview deployments and server functions, but it is intended for personal, non-commercial work. Supabase’s free plan currently includes two projects, 500 MB of database space per project, 1 GB of file storage and 50,000 monthly active users, although inactive projects may pause. ^[47] ^[48] ^[49] ^[50]

These allowances are enough for development and demonstrations, not necessarily a production business. Resume files should have strict size and type restrictions even though Supabase permits individual uploads up to 50 MB on its free plan. ^[51]

API requirements

Internal API

A clean REST-style interface might include:

```
POST    /api/profile
GET     /api/profile
PATCH  /api/profile

POST    /api/resumes
GET     /api/resumes
DELETE  /api/resumes/:id

GET     /api/jobs
GET     /api/jobs/:id
POST    /api/jobs/:id/save
POST    /api/jobs/:id/dismiss

GET     /api/applications
POST    /api/applications
GET     /api/applications/:id
PATCH  /api/applications/:id/status

POST    /api/ai/tailor-resume
POST    /api/ai/cover-letter

GET     /api/notifications
PATCH  /api/notifications/:id/read
```

Every endpoint should implement:

- Authentication
- Authorization and ownership checks
- Input validation
- Rate limiting
- Consistent errors
- Pagination
- Logging without exposing resumes or tokens
- Protection against duplicate records
- Database transactions where multiple records change together

External APIs

Need	Initial approach
Job listings	Use a developer job API, open company feeds, or a small administrator-imported dataset
AI generation	Gemini free tier or a locally hosted Ollama model
Resume extraction	Server-side PDF/DOCX parsing libraries

Need	Initial approach
Email notifications	Resend or another transactional email provider
Google sign-in	Supabase OAuth
Gmail tracking	Postpone until after the MVP
Employer-form automation	Postpone until after the MVP
Location autocomplete	Use a static location list or OpenStreetMap-compatible service carefully
Analytics	PostHog or simple internal event records

Adzuna provides programmatic access to job advertisements and normalized job information, but its terms and attribution requirements should be checked before displaying listings. Google's Gemini Developer API offers limited free access to selected models, making it an option for controlled prototype usage. ^[52] ^[53]

Security requirements

This application processes resumes and employment history, so security is a core requirement rather than a later enhancement.

- HTTPS everywhere
- Secure, HTTP-only session cookies
- Row-level security on every user-owned table
- Private resume-storage bucket
- Short-lived signed file URLs
- Server-side authorization on every mutation
- File extension, MIME type and size validation
- Malware-scanning placeholder or administrator quarantine workflow
- API rate limiting
- CSRF protection where applicable
- Protection against SQL injection through parameterized queries
- Output encoding against cross-site scripting
- Secrets stored only in server environment variables
- No service-role database key in browser code
- AI prompts protected against instruction injection from job descriptions
- Minimal logging of personally identifiable information
- Account deletion and data export
- Consent before sending documents to an AI provider
- Privacy policy and retention rules
- Regular dependency updates

- Database backups before public launch
- Separate development and production environments

Sprout's production architecture illustrates the additional obligations introduced by Gmail access: read-only OAuth scopes, restricted email selection, encrypted tokens, isolated encryption keys and deletion of Gmail-derived data when an account is removed. Gmail integration should therefore not be part of a first zero-budget version. ^[45]

Auto-apply warning

A true SORCE-style agent is the hardest part of the system. It must:

- Navigate many unrelated career sites
- Recognize fields and document-upload controls
- Handle Workday, Greenhouse, Lever and other ATS variations
- Answer employer questions safely
- Manage redirects and pop-ups
- Handle CAPTCHA and multifactor verification without bypassing them
- Preserve user approval and consent
- Detect failed or incomplete submissions
- Avoid duplicate applications
- Save evidence of what was submitted
- Adapt whenever employer sites change
- Respect platform terms and rate limits

That requires browser automation such as Playwright, persistent background workers, job queues, isolated browser sessions, encrypted credentials, retry logic and ongoing maintenance. Free serverless plans are poorly suited to long-running browser sessions, so the first version should open the employer's application page and let the user submit manually rather than pretending automation succeeded.

Top five MVP features

Candidate onboarding

Implement registration, authentication, candidate profile creation and one resume upload. Extract basic resume text to prefill experience, education and skills.

Why first: Every later feature depends on structured candidate information.

Job feed and search

Import jobs from one legitimate source and support keyword, location, remote/hybrid/on-site and employment-type filters.

Why second: It creates the core user loop without requiring fragile automation.

Save, dismiss and swipe

Let users save, dismiss or express interest in jobs, with duplicate prevention. The interaction can look like SORCE and Sprout without immediately triggering an external application.

Why third: It gives the project its distinctive product experience while remaining technically manageable.

Application tracker

Allow users to create an application manually, assign a status and see it in a Kanban board or timeline. Store every status change in application history.

Why fourth: This creates ongoing value even when applications happen outside your website.

Tailored application package

Generate a suggested resume summary and cover letter for a selected job. Require the user to review and edit the result, then download the documents and continue to the employer's website.

Why fifth: It demonstrates useful AI while avoiding the infrastructure, security and reliability problems of full auto-submit.

Build sequence

1. Create the Next.js interface and Supabase project.
2. Implement authentication and database security policies.
3. Build profile and resume management.
4. Import a limited set of jobs.
5. Add job search, save and dismiss actions.
6. Build the application tracker and timeline.
7. Add AI-assisted document generation behind strict usage limits.
8. Add tests, logging, privacy controls and account deletion.
9. Deploy through GitHub and Vercel.
10. Only after the MVP works, consider email synchronization, notifications and controlled browser automation.

The most practical zero-budget stack is therefore **TypeScript + Next.js + React + Tailwind on Vercel, backed by Supabase Authentication, PostgreSQL and Storage**, with Gemini or a local

model used sparingly for AI. Avoid microservices, paid job feeds, Gmail access and automatic ATS submission in the first release.



Is Baserow for database and posthog for analytics free and what are the programming languages for that?

Both Baserow and PostHog have free hosted plans. For your Next.js project, you would integrate both primarily using TypeScript/JavaScript—you do not need to program in the languages used internally to build either platform.

Cost and limits

Service	Free?	Main free allowance	Important limitation
Baserow Cloud	Yes, free forever	3,000 rows, 2 GB of storage and 2,000 monthly automation credits per workspace ^[122]	The 3,000-row limit may be restrictive for storing jobs, applications and activity records
Baserow self-hosted	Open-source edition is free	No Baserow-imposed cloud row or storage limits ^[123] ^[124]	You must provide and maintain the server, PostgreSQL, storage, backups and updates
PostHog Cloud	Yes, not merely a trial	First 1 million analytics events per month are free; the allowance resets monthly ^[125]	Charges can begin if you exceed the free allowance or use other products beyond their individual allowances
PostHog self-hosted	Open-source software is available	Software can be self-hosted	You pay or provide the underlying server and take responsibility for maintenance

For your job-platform MVP, PostHog’s free cloud plan should be more than adequate. Baserow Cloud is suitable for a small prototype, but 3,000 rows can disappear quickly if jobs, applications, users, notifications and application-history events are stored separately. ^[122]

Languages you use

Baserow integration

Baserow automatically exposes REST endpoints for creating, reading, updating and deleting rows. Therefore, your Next.js application can communicate with it using ordinary TypeScript: ^[126]

```
const response = await fetch(
  `${process.env.BASEROW_URL}/api/database/rows/table/${tableId}`,
  {
    headers: {
      Authorization: `Token ${process.env.BASEROW_API_TOKEN}`,
    },
  }
);
```

```
const jobs = await response.json();
```

Recommended project languages:

- **TypeScript:** Front end, server routes and Baserow API requests
- **HTML:** Generated through React components
- **CSS/Tailwind CSS:** Interface styling
- **SQL:** Not normally required when using Baserow Cloud, although understanding relational databases remains useful
- **Python:** Not required unless you later create independent resume-processing or data-import scripts

Keep the Baserow API token in a Next.js server route or server action. Never expose a database token in browser code.

PostHog integration

PostHog provides dedicated Next.js, React and Node.js SDKs. The browser package is `posthog-js`, while server routes and server actions can use `posthog-node`. [\[127\]](#) [\[128\]](#)

```
npm install posthog-js posthog-node
```

```
import posthog from "posthog-js";

posthog.capture("job_saved", {
  jobId: job.id,
  location: job.location,
});
```

You would use:

- **TypeScript/JavaScript:** Browser analytics and server-side event capture
- **React/Next.js:** PostHog provider and hooks
- **No Python required:** Unless your backend is independently written in Python

PostHog supports many other environments, including Python, JavaScript, Node.js, Java, Go, Rust, PHP, Ruby, .NET, iOS, Android and Flutter. [\[129\]](#)

Internal languages

These are the languages used to develop the services themselves—not languages you must learn to use them.

Platform	Main internal technologies
Baserow	Python/Django backend, Vue.js front end, PostgreSQL database, plus JavaScript, TypeScript, SCSS and HTML [124]
PostHog	Python, TypeScript and Rust, with smaller amounts of infrastructure and supporting languages [130]
PostHog browser SDK	Predominantly TypeScript [131]

You only need these internal languages if you plan to modify or contribute to their open-source code.

Recommended setup

For your project, the simplest Baserow/PostHog stack would be:

```
Frontend:
TypeScript + Next.js + React + Tailwind CSS

Backend:
Next.js route handlers and server actions in TypeScript

Operational database:
Baserow Cloud through its REST API

Resume file storage:
A separate object-storage service

Analytics:
PostHog Cloud using posthog-js and posthog-node

Hosting:
Vercel Hobby
```

Baserow should not receive API calls directly from the browser:

```
Browser
↓
Next.js server route
↓
Baserow REST API
```

PostHog browser events can be sent through its browser SDK, while security-sensitive or authoritative events—such as `application_created`—should be recorded from the server.

Suggested analytics events

Start with a small event vocabulary:

- `user_registered`
- `profile_completed`

- resume_uploaded
- job_viewed
- job_searched
- job_saved
- job_dismissed
- application_created
- application_status_changed
- cover_letter_generated

Do not send resume text, names, email addresses, cover letters or other sensitive personal information as PostHog event properties. Use internal identifiers and non-sensitive categories instead.

My recommendation

Use PostHog Cloud for analytics. Its free allowance is generous and its Next.js integration is straightforward. ^[125] ^[127]

Use Baserow only for an early prototype or course demonstration. Its spreadsheet-style administration is convenient, but the 3,000-row Cloud limit and API-based data access will become restrictive for a real job platform. For a more scalable zero-budget application, **Supabase PostgreSQL is the stronger primary database**, while Baserow could still serve as a temporary administrative job-entry tool. ^[122]



1. <https://www.sorce.jobs/articles/sorce-faq>
2. <https://www.usesprout.com/>
3. <https://www.usesprout.com/features/application-tracking>
4. <https://www.usesprout.com/features/ai-job-application>
5. <https://www.sorce.jobs/compare/jobcopilot>
6. <https://www.sorce.jobs/articles/automatic-job-application>
7. <https://www.sorce.jobs/articles/ai-to-apply-for-jobs-automatically-free>
8. <https://apps.apple.com/af/app/sprout-ai-job-search/id6740011494>
9. <https://help.usesprout.com/en/collections/12978904-getting-started>
10. <https://help.usesprout.com/en/articles/13802695-see-how-applications-work-in-sprout>
11. <https://www.sorce.jobs/articles/sorce-review>
12. <https://jobcopilot.com/sprout-ai-job-search-review/>
13. <https://apps.apple.com/ng/app/sorce-ai-job-search/id6504584959>
14. <https://apps.apple.com/us/app/sorce-ai-job-search/id6504584959?see-all=reviews&platform=iphone>
15. <https://www.sorce.jobs/articles/best-mobile-app-for-job-search>
16. <https://www.aitoolnet.com/sprout>

17. <https://www.funblocks.net/aitools/reviews/sorce>
18. <https://shyft.ai/tools/sorce>
19. <https://www.productcool.com/product/sorce-2>
20. <https://www.autoapplier.com/blog/tinder-for-jobs>
21. <https://sprout.hr/platform>
22. <https://test.sproutsai.com/platform-overview/>
23. <https://www.extruct.ai/hub/sorce-jobs/>
24. <https://employer.usesprout.com/>
25. <https://www.sorce.jobs/>
26. <https://apps.apple.com/us/app/sorce-ai-job-search/id6504584959>
27. <https://apps.apple.com/us/developer/sorce/id1834860654>
28. <https://apps.apple.com/in/app/sorce-ai-job-search/id6504584959?l=hi>
29. <https://apps.apple.com/in/app/sorce-ai-job-search/id6504584959?l=ur>
30. <https://apps.apple.com/sk/app/sorce-ai-job-search/id6504584959?platform=ipad>
31. <https://apps.apple.com/vn/app/sorce-ai-job-search/id6504584959?l=vi>
32. <https://apps.apple.com/us/app/sorce-ai-job-search/id6504584959?l=ko&platform=ipad>
33. <https://apps.apple.com/il/app/sorce-ai-job-search/id6504584959?l=he>
34. <https://apps.apple.com/ph/app/sorce-get-a-new-job/id6504584959>
35. <https://apps.apple.com/us/app/sorce-ai-job-search/id6504584959?see-all=reviews&platform=undefined>
36. <https://www.sorce.jobs/articles/find-jobs-with-ai>
37. <https://www.sorce.jobs/articles/copilot-jobs>
38. <https://www.sorce.jobs/articles/best-ai-job-search-tools-2026>
39. <https://www.sorce.jobs/articles/sorce-vs-traditional-job-boards>
40. <https://www.usesprout.com/features/ai-apply>
41. <https://www.sorce.jobs/articles/swipe-right-on-jobs>
42. <https://www.sorce.jobs/articles/sorce-pricing>
43. <https://www.ycombinator.com/companies/sorce/jobs/ahouT0p-member-of-technical-staff-fullstack>
44. <https://aiapplyd.com/blog/sprout-alternatives-2026>
45. <https://www.usesprout.com/legal/privacy-policy>
46. <https://nextjs.org/>
47. <https://supabase.com/docs/guides/platform/billing-on-supabase>
48. <https://supabase.com/pricing>
49. <https://vercel.com/docs/plans/hobby>
50. <https://vercel.com/docs/plans>
51. <https://supabase.com/docs/guides/storage/uploads/file-limits>
52. <https://developer.adzuna.com/>
53. <https://ai.google.dev/gemini-api/docs/pricing>

54. https://www.sorce.jobs/jobs/senior-frontend-software-engineer-react-js-and-next-js-mindera-j1_JRdBPZzWEuJpmvGtUtYnDpoZKKNCs72SBvoBBBeCDN3LSq2XhUw
55. https://www.sorce.jobs/jobs/desarrollador-a-full-stack-silent4business-j1_DZBFRP-spX9xDmVS98EBtBVTGTqgvvC_b9xgoXVJwiU6V1A3cQ
56. https://www.sorce.jobs/jobs/front-end-software-engineer-react-nextjs-hang-j1_jz_jCDvweV6Gnx2x7HdCOMo3opSnBttB4UWJaliExUz4-Bg-
57. https://www.sorce.jobs/jobs/full-stack-software-engineer-sf-rfs-group-j1_i3VWYKhHPb46H6v4jctToHa7uQz9mIkZcPbfnj6DtEGOuUzgig
58. https://www.sorce.jobs/jobs/sr-web-platform-engineer-tradestation-j1_0L_yKbcEc6sGEHNbe04rSjQnGLKFOHpEzmSK6nHfh5IzRZwPdA
59. https://www.sorce.jobs/jobs/senior-software-engineer-full-stack-javascript-able-j1_L1oOreN7vTjAlI7xJ4AwDXeiELPDUwBPNjta82f2AljqKGWTVa
60. https://www.sorce.jobs/jobs/ai-tech-lead-zurich-j1_ztRFRY47QLCrzPHjiwF3EI98hH4KD4wSrAGoikpCKY56GNpY_Q
61. <https://www.usesprout.com/drafts/careers/hex-eng-002>
62. <https://www.sorce.jobs/>
63. <https://www.sorce.jobs/resume/full-stack-engineer>
64. <https://github.com/developedbyed/sprout-scribble>
65. <https://www.sorce.jobs/reviews/sprout>
66. <https://www.usesprout.com/>
67. <https://supabase.com/docs/guides/functions/limits>
68. <https://supabase.com/docs/guides/realtime/limits>
69. <https://supabase.com/changelog/33121-relaxing-database-size-limit-on-free-plan-0-5-gb-database-size-per-project>
70. <https://supabase.com/pricing.md>
71. <https://makerkit.dev/blog/saas/supabase-pricing>
72. <https://www.cloudzero.com/blog/supabase-pricing/>
73. <https://supabase.com/docs/guides/platform/database-size>
74. <https://vercel.com/pricing>
75. <https://supabase.com/docs/guides/storage/serving/bandwidth>
76. https://www.sorce.jobs/jobs/director-solutions-architect-vercel-j1_-yPYMHMAVwiCz7HSp5tBZDQWKzkGfb4S8hNVdsIMfd5HYk-QMA
77. https://www.sorce.jobs/jobs/senior-full-stack-ai-engineer-frontend-leaning-and-digital-j1_Q0ZR2sizaQMZCnvg2DWjwCMWoc-yj06BQMgvJeeGNpqEziXU1w
78. https://www.sorce.jobs/jobs/back-end-engineer-barstool-sports-j1_qfYDeYrNtg8FUHOm-nus_na5H8nqcXnPnxJluHmhGAyJWiYQ
79. https://www.sorce.jobs/jobs/software-engineer-eve-vercel-j1_fWDzw5-1TE2Nkp82YnAzuv9251SazbmG5cyIJOJGcuqb0q9AwQ
80. https://www.sorce.jobs/jobs/senior-software-engineer-ai-ease-inc-j1_6iFlbCzdXu2UFe_xffZ8helxM9VyKMHMF1lxRRy3Ru7v-GYvh
81. <https://www.framer.com/marketplace/plugins/supabase-sync/>

82. https://www.sorce.jobs/jobs/product-engineer-backend-developer-e2b-j1_5RslAnutiJHHXQ2I8HwCeaaEtFQdaoMfegVQMgbP00bYNFQQw
83. <https://gist.github.com/sorokinvd/45ce851c79e48e6a6bdeb1c56917ecf3>
84. https://www.reddit.com/r/framer/comments/1pl890a/connecting_supabase/
85. <https://gist.github.com/netgfx/905f9122a05e3ed579110a1828c56e34>
86. <https://www.framer.community/c/developers/framer-superbase-to-track-user-events>
87. https://www.sorce.jobs/jobs/software-engineer-execution-focused-replica-inc-j1_Rsz8oSTLnVfEWlyHQ5XvC6ptwHzphBbfM-7FydTecK-d4FEW
88. https://www.sorce.jobs/jobs/senior-backend-engineer-onramp-crypto-career-renew-j1_3V13hQ24EiqJ4aO7NhSI4zFeaEApc0nd-x00c0TA-Yslzt8J
89. https://www.sorce.jobs/jobs/senior-software-engineer-next-js-typescript-mobile-workana-j1_E5Z5h7qsVGEyLVFaYz5veSi_Ki1-HHZFU8hKgiTjWyBF4UHYg
90. https://www.sorce.jobs/jobs/ai-software-engineer-nodejs-nextjs-influur-j1_Plpl_FbZl9EliTZ5MmMtb8R1XzCWmr_aPWZF2To2Ow3kmnBe
91. https://www.sorce.jobs/jobs/senior-nextjs-web-developer-xpt-software-australia-j1_A-1tKfknv-WEpaBeZsWYEErVCYCGp_1FRjdZAZStOFYq12AgA
92. https://www.sorce.jobs/jobs/senior-front-end-software-engineer-remote-bask-health-j1_u-2fi870GEoizKcdJI4omegJ2d8xNVwS84gJeTrc5uh_BSSZHW
93. <https://www.linkedin.com/company/usesprout>
94. <https://recruiting.usesprout.com/>
95. <https://play.google.com/store/apps/details?id=ai.magnifex.prepai>
96. https://www.sorce.jobs/jobs/application-security-engineer-xai-j1_2VBwPf-czNK1vONf9L8Ajdxdw9_Ex7GKE2ONBPg-psYExmMJP
97. <https://apps.apple.com/us/app/sprout-ai-job-search/id6740011494>
98. https://www.sorce.jobs/jobs/quality-assurance-engineer-lever-j1_IINxoi8LK-uXMoSod8QGw2lqCGkA7MdFyZ9eLlsD4BwKFJDNKw
99. <https://builtwith.com/>
100. <https://trends.builtwith.com/websitelist/Quicksprout>
101. <https://trends.builtwith.com/agency/Sprout-Marketing-Group/CUSA>
102. <https://trends.builtwith.com/websitelist/Sprout-Digital-Marketing/High-Technology-Spend>
103. <https://www.linkedin.com/company/sorcejobs>
104. <https://builtwith.com/toolbar>
105. <https://www.sorce.jobs/referral>
106. https://play.google.com/store/apps/details?id=com.swypebeta.app&hl=en_US
107. <https://builtwith.com/company/support>
108. <https://developers.cloudflare.com/workers/platform/pricing/index.md>
109. <https://developers.cloudflare.com/workers/platform/limits/>
110. <https://developers.cloudflare.com/workers/platform/pricing/>
111. <https://markaicode.com/benchmarks/cloudflare-workers-scalability-benchmark/>

112. <https://developers.cloudflare.com/kv/platform/limits/>

113. <https://developers.cloudflare.com/workers/ci-cd/builds/limits-and-pricing/>

114. <https://remotive.com/remote-jobs/api>

115. <https://developers.cloudflare.com/ai-search/platform/limits-pricing/>

116. <https://blog.cloudflare.com/workers-kv-free-tier/>

117. <https://developers.cloudflare.com/changelog/post/2026-02-11-subrequests-limit/>

118. <https://developers.cloudflare.com/workers-ai/platform/pricing/>

119. <https://developers.cloudflare.com/d1/platform/limits/>

120. <https://developers.cloudflare.com/d1/platform/pricing/>

121. <https://shattered.io/cloudflare-d1-free-tier-caps-workers-64mib-2026/>

122. <https://baserow.io/pricing>

123. <https://baserow.io/user-docs/pricing-plans>

124. <https://github.com/baserow/baserow>

125. <https://posthog.com/pricing>

126. <https://baserow.io/api-docs>

127. <https://posthog.com/docs/web-analytics/installation/nextjs>

128. <https://posthog.com/docs/libraries/next-js>

129. <https://posthog.com/docs/libraries>

130. <https://github.com/PostHog/posthog>

131. <https://github.com/PostHog/posthog.js>

132. <https://baserow.io/user-docs/subscriptions-overview>

133. <https://www.trustradius.com/products/baserow/pricing>

134. <https://github.com/code/app-python-baserow>

135. <https://aiidelist.com/ide/baserow>

136. <https://railway.com/deploy/baserow-railway>

137. <https://www.everydev.ai/tools/baserow>

138. <https://hackceleration.com/labs/review/baserow>

139. <https://www.apibenchmarks.com/featureflags/posthog>

140. <https://github.com/baserow>

141. <https://toolradar.com/tools/baserow/pricing>

142. <https://github.com/bram2w/baserow>

143. <https://github.com/PostHog/posthog/wiki/Go-integration>

144. <https://github.com/PostHog/posthog/blob/master/CLAUDE.md>

145. <https://posthog.com/docs/experiments/installation/nextjs>

146. <https://preview.posthog.com/docs/libraries/js>

147. <https://posthog.com/docs/references/posthog-js>

148. <https://posthog.com/docs>

149. <https://posthog.com/docs/libraries/js>

150. <https://github.com/PostHog/posthog/blob/master/frontend/src/queries/schema/schema-general.ts>

151. <https://posthog.com/docs/error-tracking/installation/nextjs>